

伏安超充

基于 OpenHarmony 与通晓开发板的 AI 智能充电站集成方案



VOLT CHARGE

摘要

随着全球新能源汽车渗透率的持续攀升，大功率超级充电技术与智能化电网互联已成为下一代充电基础设施发展的必然趋势。然而，现有的传统充电桩基础设施在面对海量并发数据、多模态设备协同以及复杂用电负荷时，正面临着严峻的技术瓶颈。由于传统方案普遍采用碎片化、封闭式的嵌入式操作系统，导致设备之间的“信息孤岛”现象严重，跨平台互联互通成本极高。同时，现有设备普遍缺乏强大且高弹性的边缘计算支撑，无法在充电过程中对电池状态、电网负荷以及设备发热进行实时的智能化感知与风险预测，难以满足未来智慧城市与微电网对超级充电基础设施高敏捷、高安全性的严苛要求。

针对上述行业痛点与技术挑战，本项目设计并实现了“伏安超充充电桩：基于 OpenHarmony 与通晓开发板的 AI 智能充电站集成方案”。本系统立足于前沿的开源分布式操作系统 OpenHarmony，深度利用其特有的分布式软总线、设备虚拟化及原子化服务特性，从底层打通了车、桩、人、云之间的通信链路，实现了设备间动态、无缝的泛在协同。在硬件控制层，系统搭载高性能通晓开发板作为整个智能充电站的边缘控制大脑，依托其卓越的异构计算能力与丰富的外设接口，不仅保障了底层硬件驱动的高效响应，更为上层复杂业务逻辑的平稳运行提供了坚实的硬件基座。

在软硬件深度融合的基础上，本系统将边缘端 AI 智能作为核心突破口。依托昇腾开发板的强劲算力，系统在边缘侧部署了轻量化 AI，能够为客户与管理员提供卓越的相关建议，并可以对电池健康度（SOH）在线诊断，实现了由传统“被动防御”向“主动安全预测”的跨越式转变。同时，为了验证本套完整解决方案的创新性与可用性，项目完全基于高仿真模拟充电桩进行了全流程的工程化开发与实际部署，真实还原了真实充电桩中的复杂工况，确保了测试结果的客观性与工业参考价值。

全面的系统测试与实验数据表明，该方案在数据吞吐量、边缘端响应时效以及长期运行的系统鲁棒性上均表现优异，充分证实了鸿蒙生态与高性能边缘硬件结合在工业级能源场景中的落地可行性。本项目的成功构建，不仅成功攻克了传统充电桩互联难、不智能的顽疾，为新能源超级充电领域提供了一套高可靠、可扩展的样板工程，更从架构设计、边缘智能到分布式协同等多个维度，为未来 AI 智能充电桩的应用落地与产业生态构建提出了全新的范式。

第一部分 作品概述

1.1 功能与特性

本项目实现了基本的充放电功能，在此基础上，硬件系统会对充电桩的环境温度、湿度进行检测。同时充电枪实现了实时插入检测，当充电器被用户插入设备时，系统将识别并给出提示，当用户未插枪启动充电，系统会阻止该行为。

插枪后，系统会对充电车辆发起协议握手，系统会区分快充和慢充车辆，并匹配其适宜的电压平台。当用户完成支付，电桩接收到服务器的指令后，系统将开启充电。系统会持续检测所使用的充电枪的电压、电流、温度等信息，同时接收来自充电车辆反馈的温度、电量等参数信息。当用户强行拔出充电枪或发生高温等情况，充电桩会识别并立即断电。

本充电桩还实现了光伏与储能，充电桩携带了一块储能电池，主系统依托UPS模块可在市电掉电时无缝切换至储能供电，同时可主动切换充电枪的供电路径。储能电池由光伏管理模块进行充电，该模块会对光伏板的最大功率点进行追踪，发电后向储能电池进行充电，当太阳足够时，模块会自动使光伏板直接介入供电。

1.2 应用领域

本方案首先适用于城市智慧公共超充网络。在商业综合体、高速服务区等高并发场景下，方案利用 OpenHarmony 分布式架构将多台充电桩虚拟化协同，依托通晓开发板进行本地 AI 功率动态编排，在边缘端自主优化多车排队与电能调度，能显著提升公共充电站的运营效率，缓解电网高负荷冲击。

其次，本方案可深度应用于微电网与车网互动（V2G）场景。依托边缘端 AI 算法，系统能够实时预测电网负荷，并结合分布式软总线技术，实现光伏跟踪、

储能电池充放电与充电桩功率的毫秒级本地联动。在电网峰谷调节与清洁能源消纳中，该方案能确保充电站自主维持局部功率平衡，为绿色能源互联网提供底层技术支撑。

最后，该方案广泛适用于工业园区及封闭场景的无人化充换电系统。在港口、矿山或物流园区等恶劣工业环境下，系统凭借高确定性时延与丰富的工业接口，能完美对接无人驾驶重卡、AGV 及充电机械臂，实现车、桩、多传感器间的近场安全协同与故障主动预警，全面赋能工业场景的绿色与无人化转型。

1.3 主要技术特点与创新点

插入检测与通信：本项目创新地提出对 **Type-C** 接口进行魔改，将 **CC** 引脚的分压设计巧妙地转变为本项目的插入检测功能，**D+**、**D-**设计为串口的 **RX**、**TX** 引脚，使一杆充电枪用一条线实现了集充电、通信、插入安全检测。

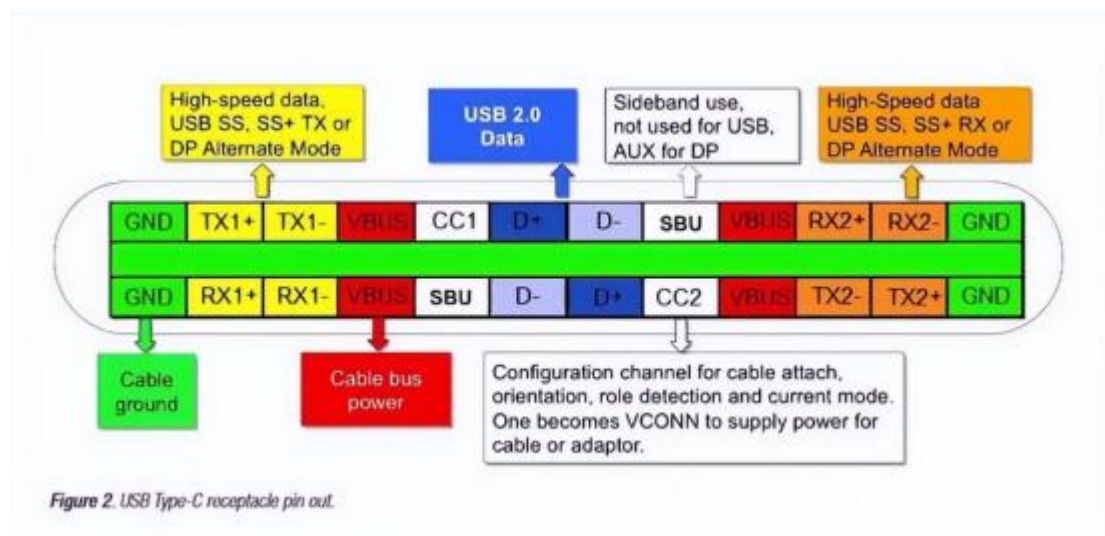


图 1 Type-C 引脚说明图

光伏储能与 UPS：充电桩通常设立在露天环境中，天然地适合设计为光伏发电的场地，本项目创新地将光伏发电、储能、UPS 有机地进行整合，顺应国家号召，实现绿色用电。光伏板发出的电被我们的光伏管理模块适时供应或为储

能电池充电，从储能电池出来的电接入到了我们的 UPS 模块，当市电掉电时，UPS 模块将无缝切换到储能供电，管理员后台也可以主动对 UPS 发起切换。

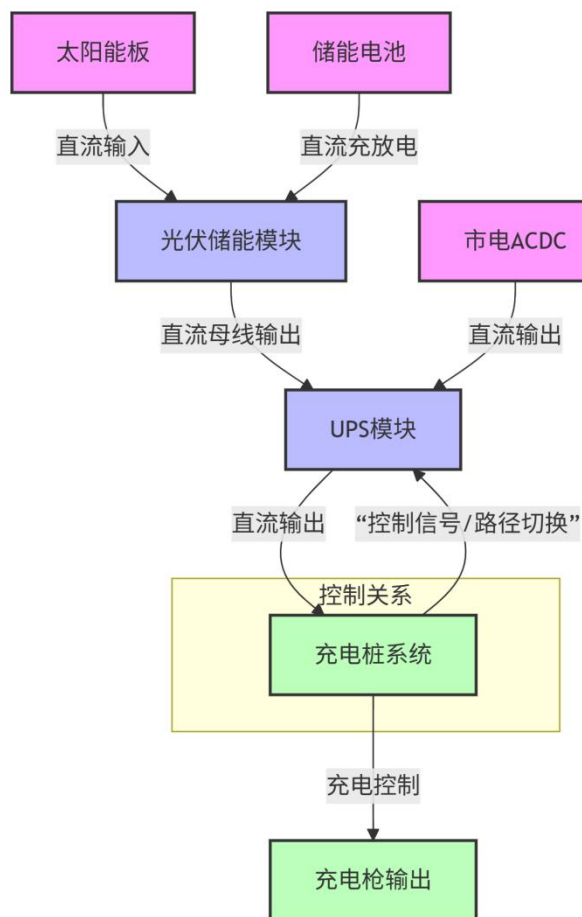


图 2 电源架构图

前卫的 APP 设计：告别沉闷的设计，本项目在 UI/UX 上下足了功夫，通过对华为鸿蒙的深度研究，我们实现非常美观大气的 UI，随处可见的毛玻璃视效，统一的色彩管理，无不抓住每一位使用者的眼球。



图 3 APP 页面众览

1.3 设计流程

本项目跨越全栈，自研比例极高，项目的最初先进行了底层硬件的制作：从项目功能规划，再到原理图、PCB 设计，最终打样生产，焊接，但这只是开始，收到的每一版 PCB 总会有不同的问题，于是又经过了无数次迭代，而这样的 PCB 我们制作了 5 块。

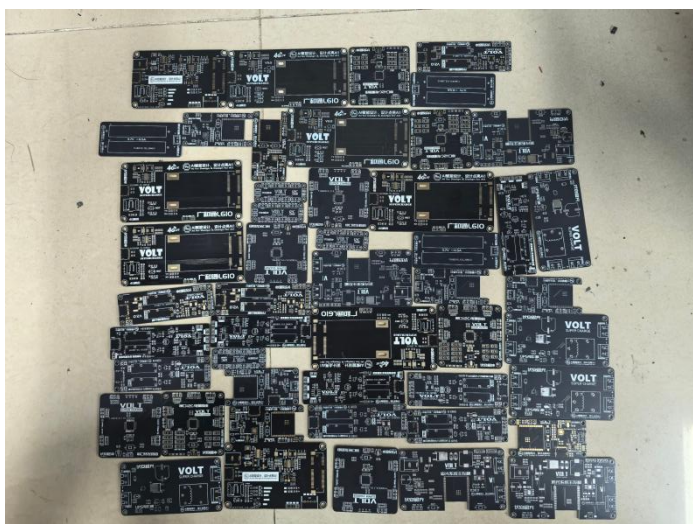


图 4 被“优化”下来的 PCB

完成硬件设计后，我们开始了南向的代码设计，我们充电桩使用了通晓开发板，小车使用了星闪模块，尽管都搭载了鸿蒙 LiteOS，但开发方式和项目架构乃至 API 都大相径庭，这在开发中造成了不少困惑，攻克了无数个日夜。

当南向没问题后，我们开始搭建后端，租赁云服务器，搭建 Ubuntu 以及 EQMX，并开始了服务器代码的编写，并制定了协议文档便于通信，通过 Python 我们很快完成了数据库和 API 的设计。

VOLT伏安超充 MQTT协议V1.4 (2026.7.4修订)							
指令内容	指令json	指令value	方向	Topic	作用区域	是否入数据库	备注
充电桩1控制状态	GunCtrlSta	1: 用户刷卡支付; 启动充电3: 用户主动停止充电4: 管理员后台支付停止充电	南向	Control	用户	否	
充电桩2控制状态	Gun2CtrlSta	1: 用户刷卡支付; 启动充电3: 用户主动停止充电4: 管理员后台支付停止充电	南向	Control	用户	否	
所有充电桩停止充电	StopAllChar	1: 停止所有充电	南向	Control	管理员	否	
充电桩1充电控制	PowerGunCtrl	1: 切换充电桩2; 切换储能供电	南向	Control	管理员	否	
充电桩1用户名	Gun1UserName	"name": 用户登录名	南向	NorthData	用户	否	
充电桩2用户名	Gun2UserName	"name": 用户登录名	南向	NorthData	用户	否	
充电桩1用户余额	Gun1UserBal	xxx: 余额	南向	NorthData	用户	否	默认保留一位小数, 乘以10以整形形式发送, 南向再除以10, 如12.5, 北向发送125, 南向收到后除以10得到
充电桩2用户余额	Gun2UserBal	xxx: 余额	南向	NorthData	用户	否	默认保留一位小数, 乘以10以整形形式发送, 南向再除以10, 如12.5, 北向发送125, 南向收到后除以10得到
充电桩1主动停止	Gun1Stop	1: APP主动停止2: 充满停止3: 高温停止4: 火灾停止5: 接触停止6: 触摸屏停止	北向	Control	用户	否	
充电桩2主动停止	Gun2Stop	1: APP主动停止2: 充满停止3: 高温停止4: 火灾停止5: 接触停止6: 触摸屏停止	北向	Control	用户	否	
充电桩1电池容量	Gun1BatCap	xxxx: 电池容量大小	北向	SouthData	用户	否	默认不包含小数
充电桩2电池容量	Gun2BatCap	xxxx: 电池容量大小	北向	SouthData	用户	否	默认不包含小数
充电桩1充电协议	Gun1Agt	0: 无协议1: 普通速度充电2: 超快充	北向	SouthData	用户	否	
充电桩2充电协议	Gun2Agt	0: 无协议1: 普通速度充电2: 超快充	北向	SouthData	用户	否	
充电桩1当前状态	Gun1State	0: 空闲1: 已插枪2: 充电中3: 高温预警4: 火灾预警	北向	SouthData	用户	否	
充电桩2当前状态	Gun2State	0: 空闲1: 已插枪2: 充电中3: 高温预警4: 火灾预警	北向	SouthData	用户	否	
充电桩1温度	Gun1Temp	xx.x: 充电桩1温度	北向	SouthData	用户	否	默认保留一位小数
充电桩2温度	Gun2Temp	xx.x: 充电桩2温度	北向	SouthData	用户	否	默认保留一位小数
充电桩1环境温度	SysTemp	xx.x: 充电桩环境温度	北向	SouthData	用户	否	默认保留一位小数
充电桩2环境温度	SysHumi	xx.x: 充电桩环境温度	北向	SouthData	用户	否	默认保留一位小数
充电桩1当前放电电压	Gun1Vol	xx.x: 当前放电电压	北向	SouthData	用户	否	默认保留一位小数
充电桩2当前放电电压	Gun2Vol	xx.x: 当前放电电压	北向	SouthData	用户	否	默认保留一位小数
充电桩1当前放电电流	Gun1Cur	xx.x: 当前放电电流	北向	SouthData	用户	否	默认保留一位小数
充电桩2当前放电电流	Gun2Cur	xx.x: 当前放电电流	北向	SouthData	用户	否	默认保留一位小数
充电桩1当前已充入电量	Gun1CharSec	xx.x: 当前已充入电量	北向	SouthData	用户	是	默认保留一位小数
充电桩2当前已充入电量	Gun2CharSec	xx.x: 当前已充入电量	北向	SouthData	用户	是	默认保留一位小数
充电桩1当前被充入电量	Gun1Soc	xxxx: 当前被充入电量	北向	SouthData	用户	否	默认不包含小数
充电桩2当前被充入电量	Gun2Soc	xxxx: 当前被充入电量	北向	SouthData	用户	否	默认不包含小数
充电桩1当前被充入温度	Gun1CharTemp	xx.x: 当前被充入温度	北向	SouthData	用户	否	默认保留一位小数
充电桩2当前被充入温度	Gun2CharTemp	xx.x: 当前被充入温度	北向	SouthData	用户	否	默认保留一位小数
充电桩1当前被充入充电健康	Gun1Sch	xxxx: 当前被充入充电健康	北向	Control	用户	否	默认保留一位小数

图 5 本项目使用到的协议文档

最终，APP 的开发开始了，我们搜索了大量相关的设计案例，并实际下载体验了众多充电桩 APP，为了使 APP 更贴近现实，更具备落地意义，我们在无数个日夜的奋斗下，最终推出了美观而又便捷的伏安超充 APP。

第二部分 系统组成和功能说明

2.1 整体介绍

整套系统架构本质上是一个基于“云-边-端”协同的智能充电桩控制系统。系统在设计上实现了业务逻辑与底层控制的解耦，并引入了绿色能源与冗余供电机制。

在智能算力与通信层面，系统通过部署在腾讯云上的 MQTT 服务器与 API 服务器进行远程管理与业务调度。设备本地则搭载了具备 AI 算力的昇腾开发板作为边缘计算核心，用于本地化的业务处理与智能分析。边缘核心通过广和通 L610 4G 通信模块，利用 4G 网络与云端进行高频、低延迟的数据交互，构成了系统的数据通信骨干。

在底层控制与数据路由层面，系统采用了模块化的设计架构。G0B2 串口路由模块作为核心的数据调度枢纽，通过数据透传机制，将上层边缘计算板卡的控制指令高效、透明地分发至通晓开发板（系统主控）以及最末端的充电枪执行机构。这种设计弥补了通晓开发板串口数量不足的问题，既保障了底层控制的实时性，也便于后续系统的升级与维护。

在动力与电源管理层面，系统具备高可靠性的双路冗余供电能力。系统同时引入了光伏储能模块（太阳能供电）与 AC-DC 电源模块（市电输入），两路电源共同接入 UPS 电源管理模块。该模块不仅能实现绿色能源与传统市电的互补利用，还能在市电异常时提供不间断电源保障，输出稳定的直流电与动力控制电源，确保通信、边缘计算、主控核心以及充电枪系统的连续、安全运行。

总而言之，本系统通过云端数字化管理、边缘 AI 计算、灵活的数据透传路由以及光储互补的冗余供电设计，构建了一个高效、稳定且具备高扩展性的智能化充电控制解决方案。

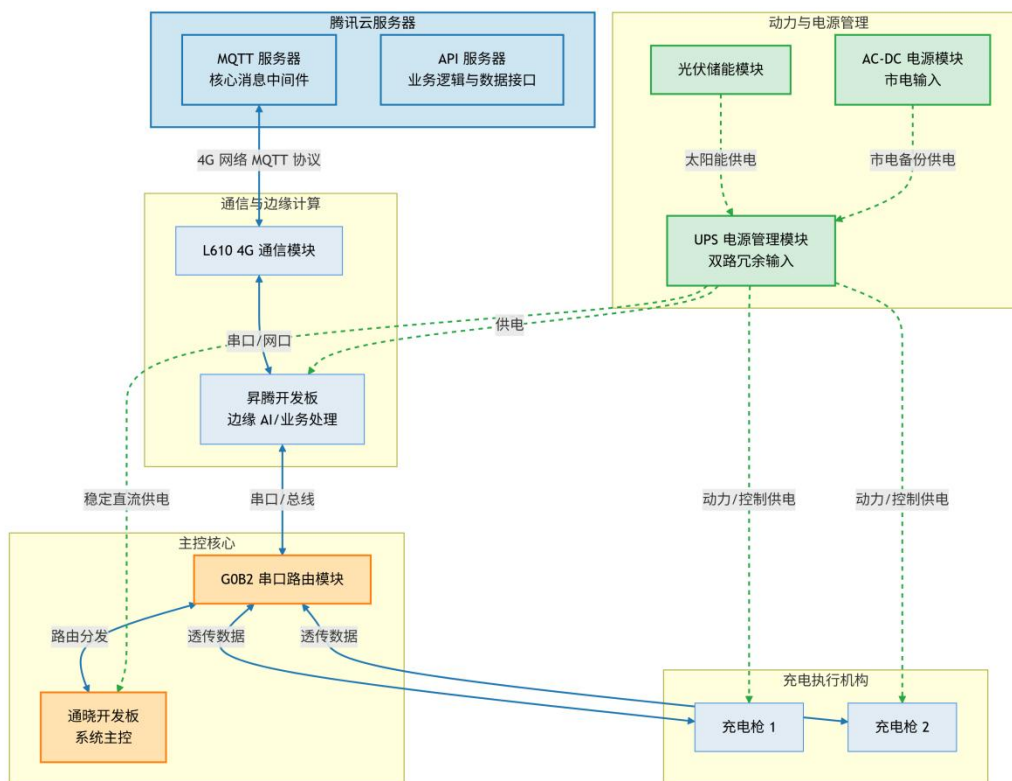


图 6 本项目总体架构图

2.2 硬件系统介绍

充电枪控制模块：每一杆充电枪使用了一块该充电枪控制板，本模块主要基于 MP2225G，通过通晓开发板控制的两路继电器为充电设备提供了高压快充和低压慢充两种充电平台，基于我们魔改的 TypeC 接口，充电小车将通过本模块与主控进行通信，在完成快充协议握手后，主控将为其匹配合适电压平台，该模块还集成了电荷、电压、电流、温度等多参数的检测，基于软件的算法，能实现对充电电池的健康度检测。

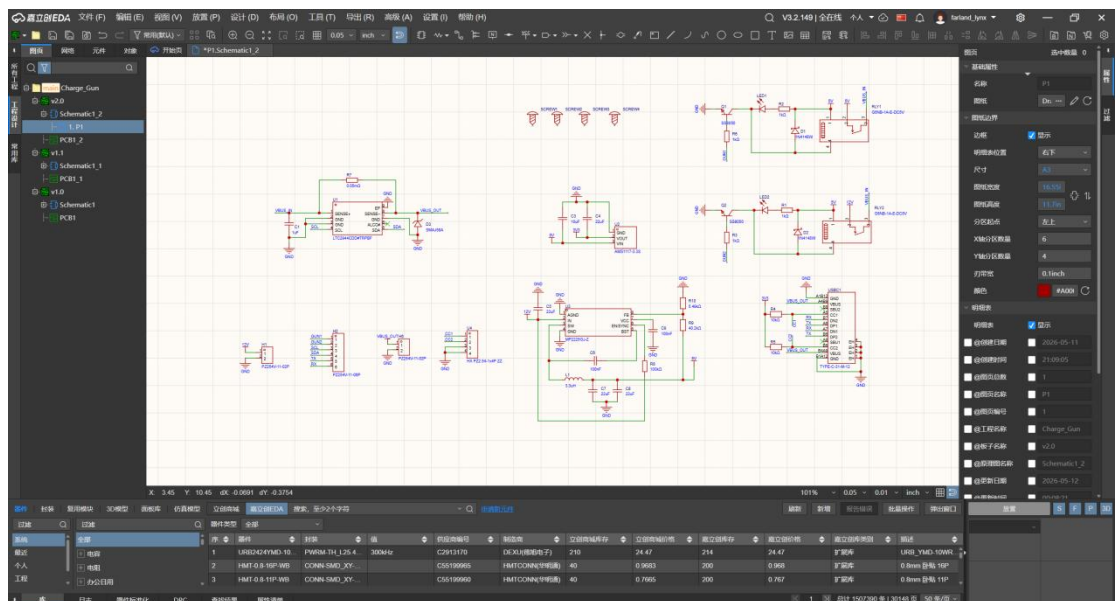


图 7 充电枪控制模块原理图

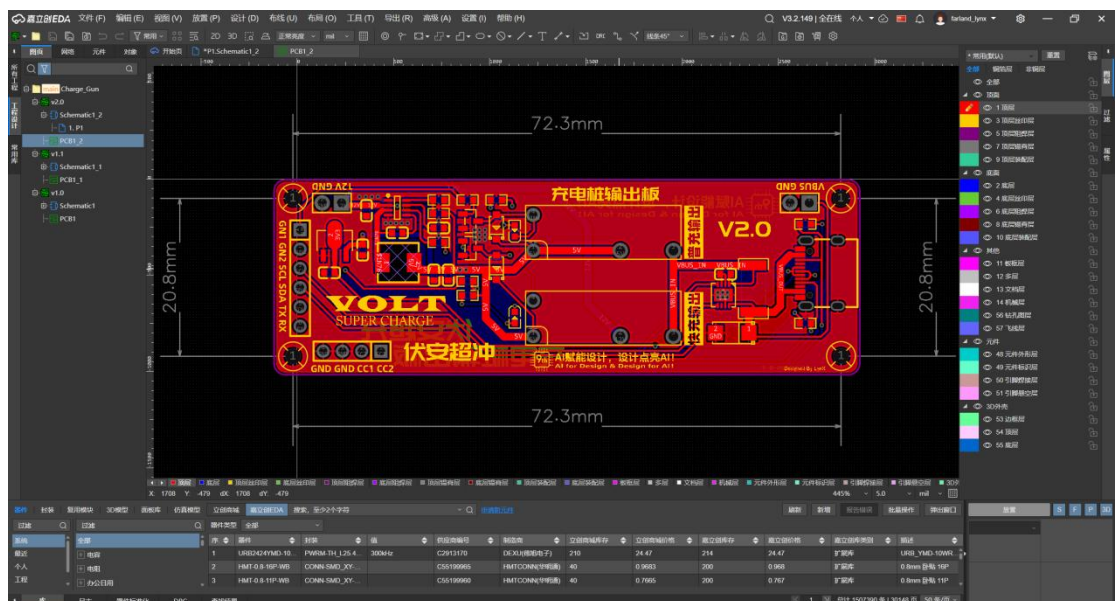


图 8 充电枪控制模块 PCB 图

光伏储能模块：本模块基于 CN3863 进行光伏发电，该芯片会对太阳能板的 MPTT 进行追踪，使得获得最佳发电功率。该芯片同时对储能电池进行充电，并适时在阳光充足的时候，直接为后端供电。模块还设计了对太阳能发电电流电压的采集，电池电压、电流、电量的采集，以及本模块温度的采集。

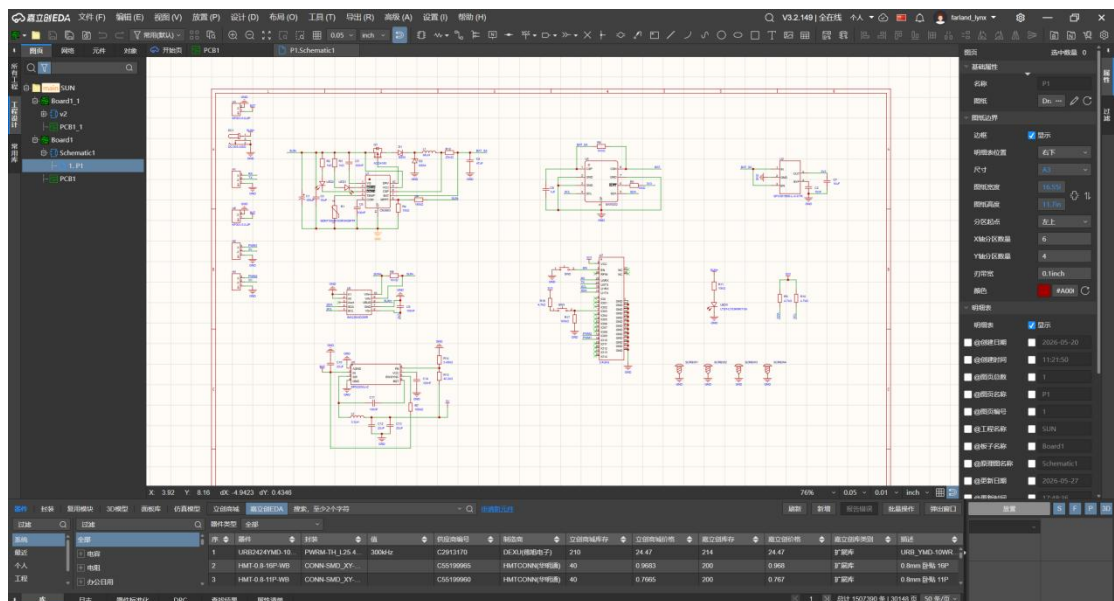


图 9 光伏储能模块原理图

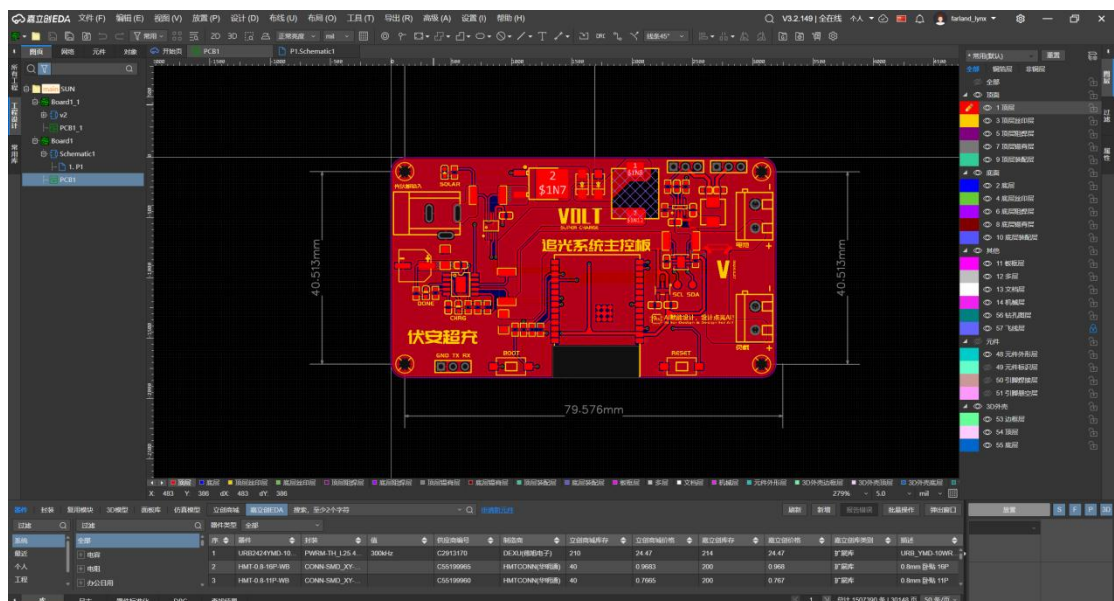


图 10 光伏储能模块 PCB 图

UPS 模块：本模块基于 LTC4412 进行设计，实现了传统 UPS 无缝切换电源的基本功能，本项目创新的使用一个单刀双掷转换型继电器，使得主控可以控制充电枪供电的路径，便于管理者使用绿色能源来节省成本。

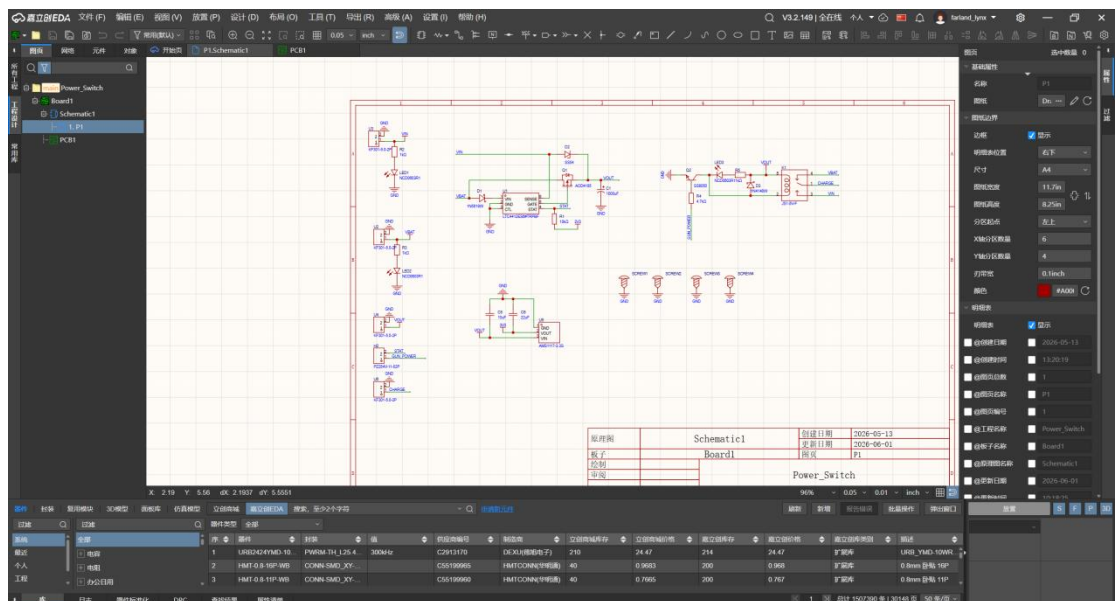


图 11 UPS 模块原理图

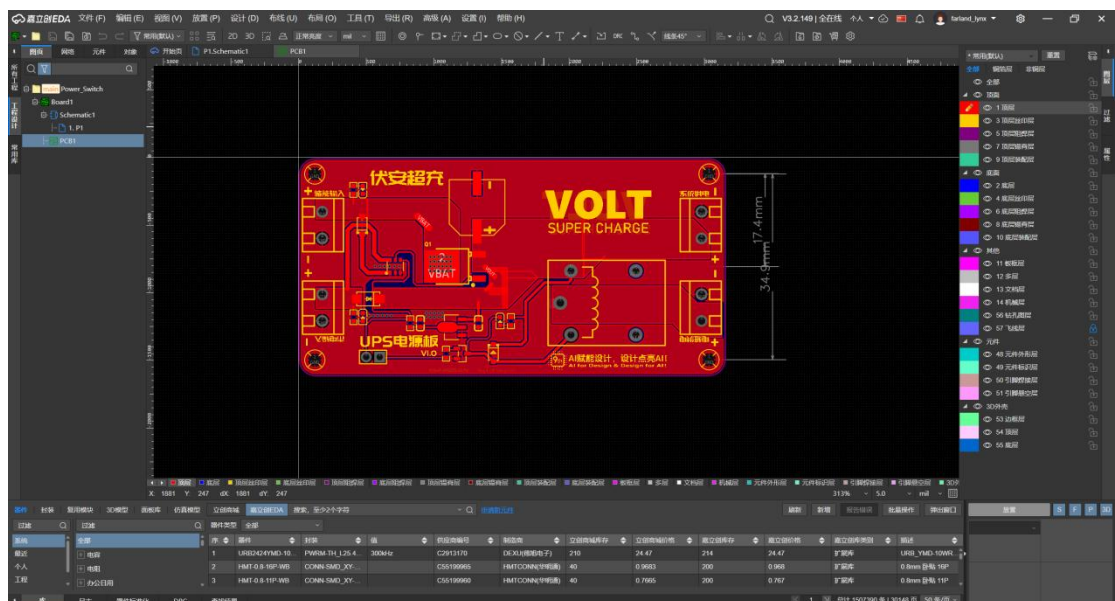


图 12 UPS 模块 PCB 图

模拟充电小车模块：为了正确检验本项目充电桩的功能，特设计了一款充电小车，基于 BQ24195 芯片，使得本模块支持了最大 18 伏的快充平台，并通过 MAX17048 芯片，可精准获取电池的电压与电量，通过我们魔改的 TypeC 接口与充电桩进行握手以及通信回传数据。

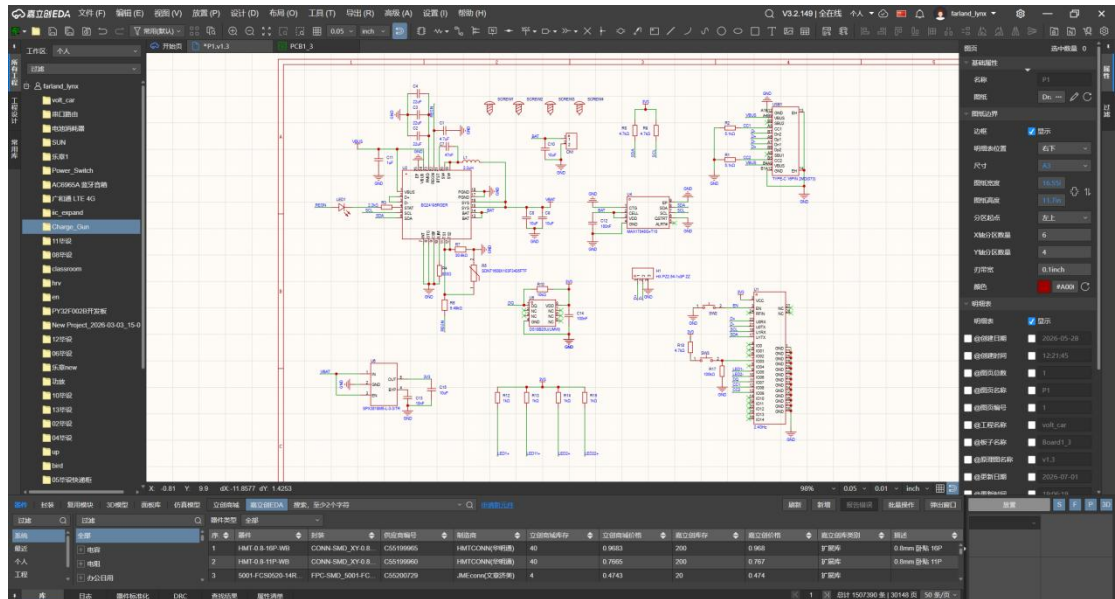


图 13 模拟充电小车模块原理图

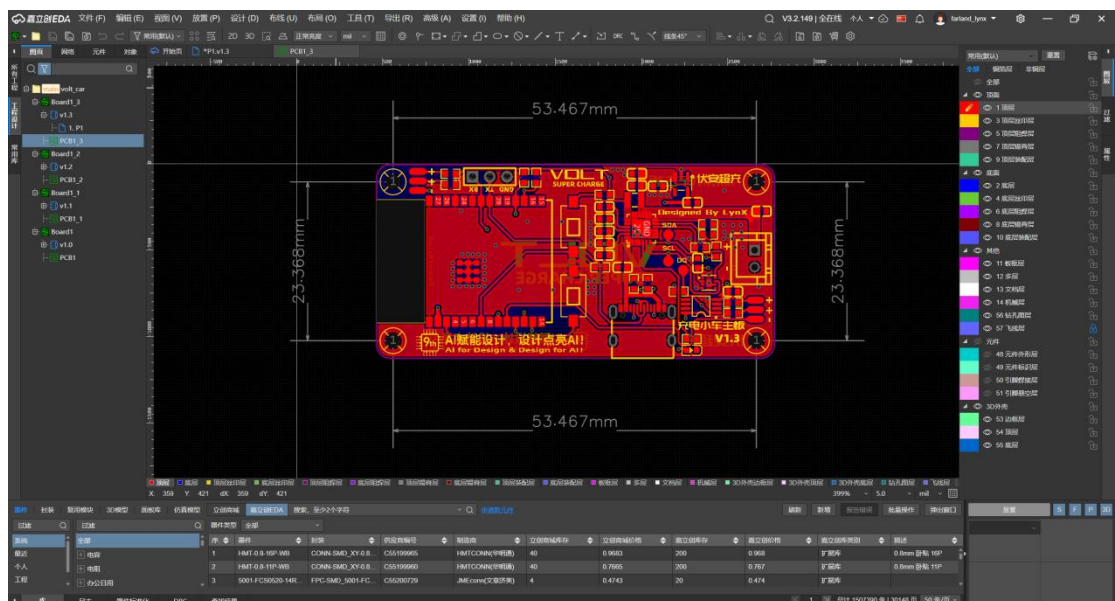


图 14 模拟充电小车模块 PCB 图

串口-I2C-电源路由模块：由于本项目大量地使用到了串口和 I2C 设备，知晓开发板现有的串口资源不足以支撑本项目，因此特设计了本模块，使用 g0b1 芯片实现了串口 1 转 5 的功能，并为引脚有限的 I2C 和电源引脚做了引出。

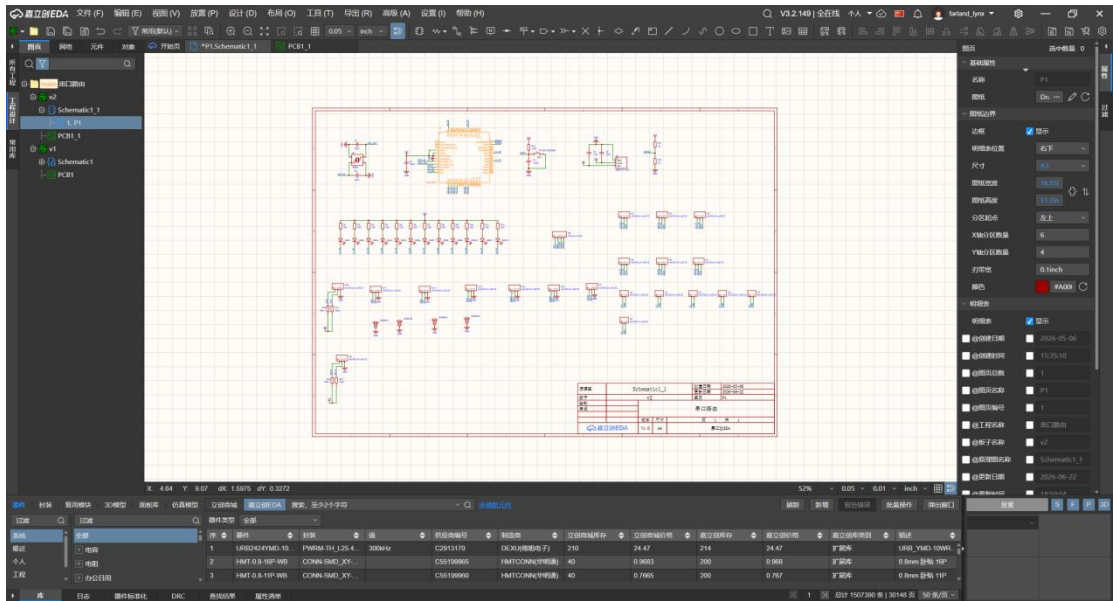


图 15 串口路由模块原理图

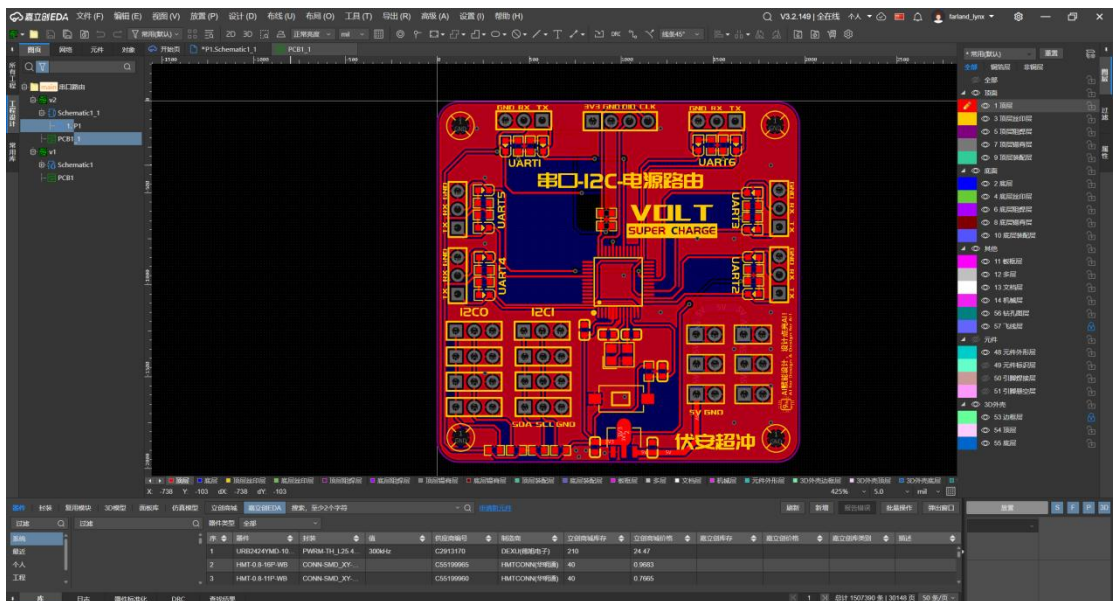


图 16 串口路由模块 PCB 图

华为昇腾开发板：本项目使用华为昇腾开发板作为边缘 ai 算力设备为本项目充电桩提供端到端算力，无需依赖云端，同时也搭配 L610 作为本项目网络结

构的一环。



图 17 华为 Atlas 200I DK A2

广和通 L610：本项目使用了广和通 L610 4G 模块作为通信的设备，挂载在昇腾开发板上作为网卡，将南向数据和北向控制指令进行双向传输

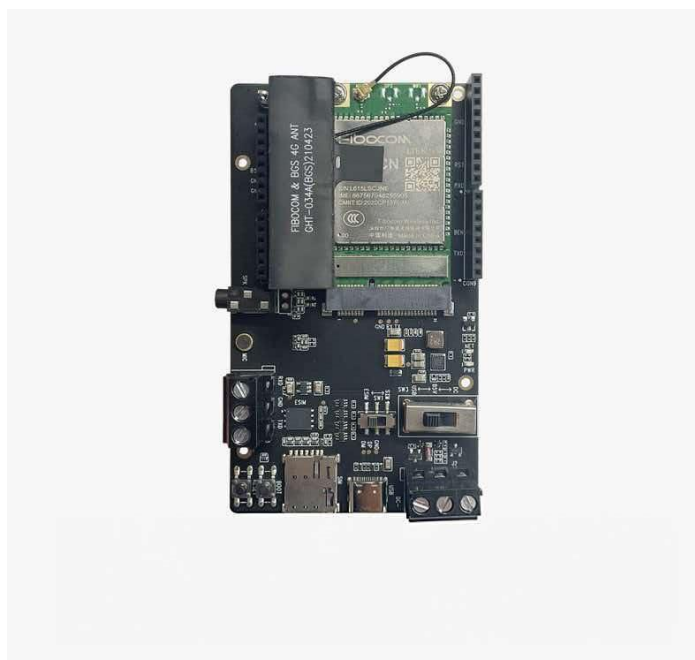


图 18 广和通 L610 模组

ACDC: 本项目采用了一个 12V/4.2A 的 ACDC 作为本项目市电电源,为所有模块进行供电。



12V

4.2A

图 19 ACDC 模块

2.3 软件系统介绍

本项目整个技术架构同样遵循严谨的分层设计，主要分为四个核心图层：在云端层，系统构筑了整体的数据与业务中枢。云服务器采用 **Python** 作为核心开发语言，并基于 **FastAPI** 框架构建高并发的 **API** 服务，整体在 **PyCharm** 环境下进行开发。该层负责向下游的客户端与硬件端提供数据接口与协议解析。

在 **APP** 客户端层，系统面向用户端提供了原生的智能交互体验。该端基于 **Harmony 6.0.1 (API21)** 系统进行开发，采用鸿蒙生态的 **ArkTS** 语言进行编写，开发工具则统一使用官方的 **DevEco Studio**。客户端通过 **4G/HTTP** 协议与云端层进行双向通信，实现用户对充电状态的远程查看与控制。

在充电桩主控层，即前述硬件系统中的底层控制大脑。该层以通晓开发板为核心硬件，搭载轻量级物联网操作系统 **LiteOS**，采用 **C** 语言 进行底层驱动与业务逻辑的编写，并在 **VS Code** 环境下完成代码调试。该主控层向上通过 **4G/MQTT** 协议与云端层保持高频的消息同步。

在被充电小车层，作为系统的物理受控末端。该层基于 **WS63** 进行开发，运行定制化的 **LiteOS-Fbb** 操作系统，同样基于 **C** 语言 进行底层控制算法的编写，使用 **HiSpark**（适配版） **IDE** 进行开发。

总结而言，这幅架构图呈现了一个以鸿蒙生态（**HarmonyOS/LiteOS**）为端侧核心，以 **Python/FastAPI** 为云端纽带的完整物联生态闭环。通过 **HTTP** 与 **MQTT** 双协议的配合，实现了从用户手机端、云端服务器、充电桩主控到末端受控小车的全链路打通。



图 20 软件架构图

在通晓开发板的业务上，整套系统的嵌入式软件设计采用了经典的状态机架构，核心逻辑围绕着“安全”与“云端协同”展开。系统在开机后，首先会对底层的所有传感器进行初始化与校准，随后便进入常态化的主循环。在主循环中，充电桩会扮演一个定时上报者的角色，周期性地向云端服务器传送当前的电压、电流、温度等环境与物理参数，以此向云端证明设备处于健康且在线的状态。与此同时，系统在本地会高频次地轮询监听充电枪的插拔状态，一旦检测到有设备插入，便会立刻打破当前的静止循环，进入下一个业务阶段。

插枪之后的首要任务是建立本地的信任机制，即协议握手。主控系统会与接入的设备进行通信协商，校验其身份与电气参数。如果握手因为协议不匹配或硬件故障而失败，系统不会盲目等待，而是会立即将失败通知上报给云端，并安全地退回到主循环中，防止程序死锁。如果握手成功，系统也不会直接开始供电，而是会进入一个“安全挂起”的状态，静静等待云端服务器下发允许充电的控制指令。这种设计确保了云端在计费 and 远程调度上的最高控制权。

在接收到云端的开始指令后，系统才会正式驱动底层执行机构进行充电。在充电期间，程序会进入一个高强度的闭环监测状态，对充电过程中的动态指标进行毫秒级的安全监控。系统将“充电自然完成”与“突发电气异常”视为同等重要的触发条件，一旦传感器捕捉到过压、过流或者超温等安全隐患，程序会立刻切断充电回路以实现硬件保护。最后，不论是正常结束还是异常中止，系统都会

在停止充电的同时，将最终的结算数据与日志上报给云端，随后再次返回主循环，等待下一次服务。

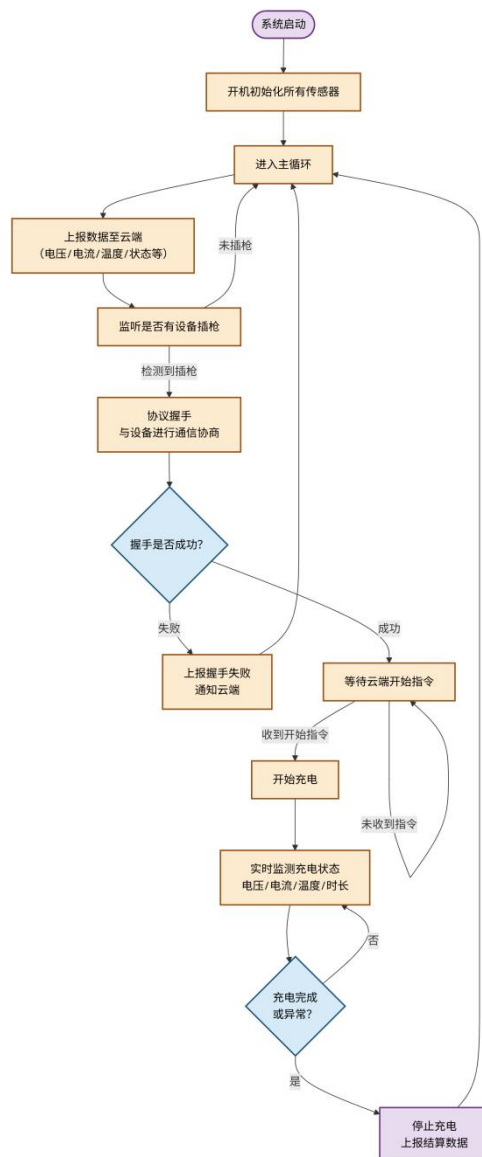


图 21 通晓软件业务软件架构图

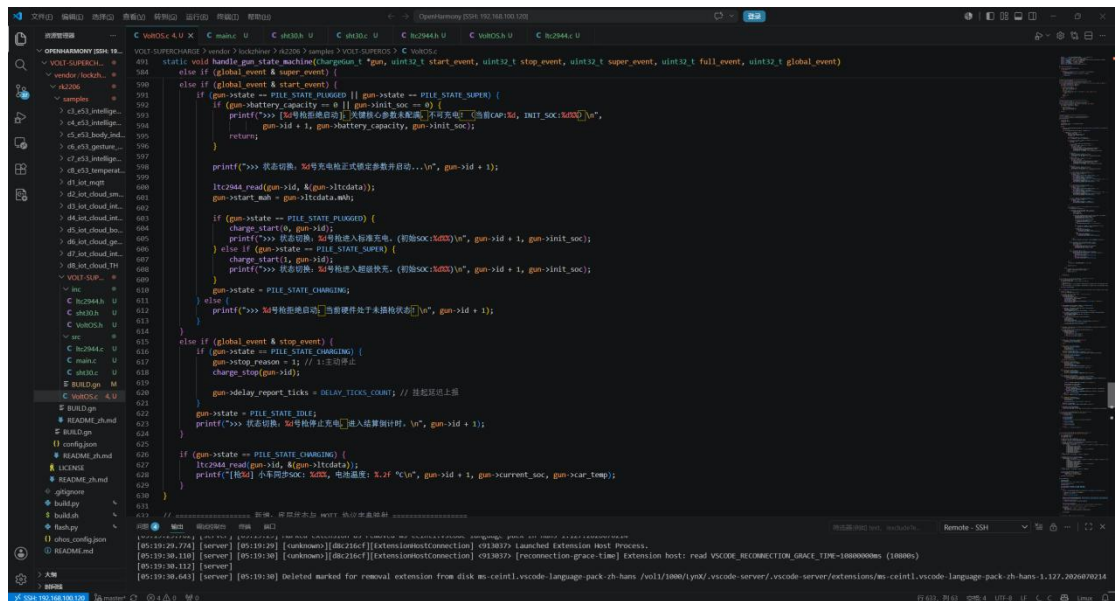


图 22 通晓软件代码截图

小车端代码，在系统启动阶段，小车主控首先执行硬件初始化，重点加载所有的 I2C 外设、传感器、电池管理芯片（BMS）以及本地的通信接口。初始化完成后，程序同样进入待机轮询状态，持续监听是否插入了充电枪。如果未检测到插入，系统将在本地保持等待。

一旦硬件层检测到充电枪插入，小车主控会主动尝试启动协议握手，与充电桩进行通信协商。如果握手不幸失败，小车端会执行本地的错误处理逻辑，即上报握手失败、记录系统日志并尝试重试，随后重新回到监听插枪的循环中。如果握手成功，小车端则进入挂起状态，等待充电桩下发正式的开始充电指令。

当接收到充电桩的开始供电信号后，小车进入正式的充电执行阶段。在这个过程中，小车主控会持续回传自身的实时数据，包括当前的电压、电流、温度、剩余电量（SOC）以及充电时长等关键参数。在这个循环监控中，系统会不断判断“电池是否充满”。如果未充满，小车将维持数据回传与充电状态。

一旦电池管理芯片判定电池已充满，小车端会主动发出“充满信号”，通知充电桩停止供电。随后，小车自身停止输出数据，正式结束本次充电会话，程序安全地返回到最初的监听插枪状态，等待下一次充电。

整套逻辑体现了端侧设备在物联网充电场景下的主动安全防护意识，通过 I2C 实时读取 BMS（电池管理系统）数据，并由小车端最终决定“何时因满电而断开”，从而有效保护了电池的寿命与安全。

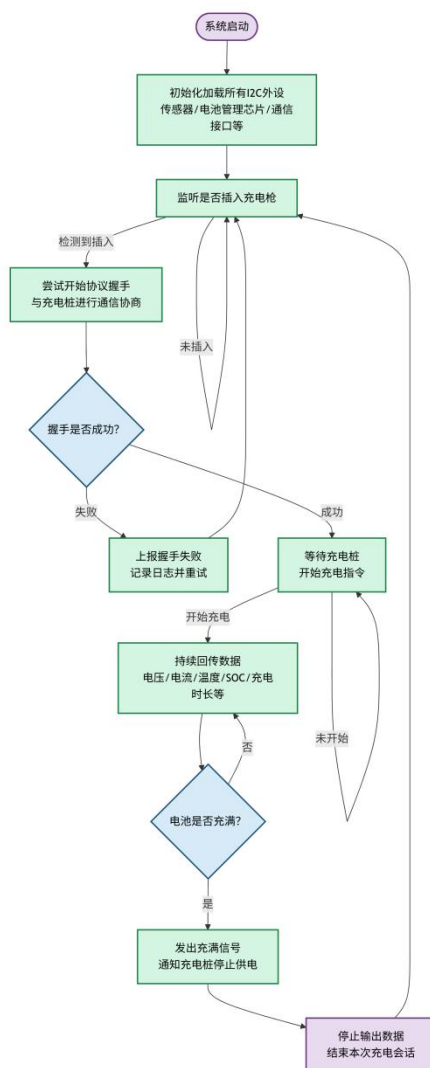


图 23 小车软件业务软件架构图

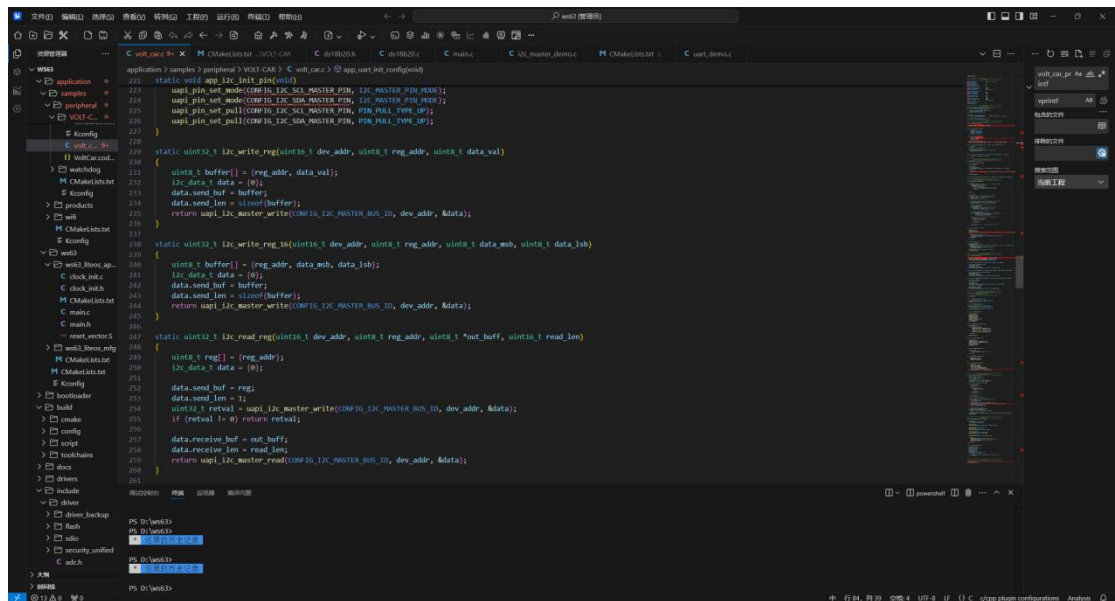


图 24 小车软件代码截图

鸿蒙客户端 APP 整个应用采用了主流的移动端门面模式设计，以一个底部的核心导航栏为纽带，向下延伸出四大核心业务板块，并针对充电这一主业务流程设计了清晰的线性跳转闭环。

应用在启动后，首先加载并渲染作为独立导航栏页面的 **Home** 页面。该页面承载了底部的四个核心图标，作为全局常驻的导航控制中枢。用户可以通过点击底部不同的图标，在四个平级的核心业务页面之间进行无缝切换。

第一个是核心的“**HomePage** 主页”。这里是用户寻找充电资源的入口，集成了充电桩地图定位、附近站点推荐以及高频功能的快捷入口。围绕着主页，应用展开了最核心的充电业务流：当用户在地图上点击某个具体的充电桩时，应用会独立跳转至“**Details** 充电桩详情页”以展示该站点的具体参数；用户在详情页确认无误并点击开始充电后，系统会进一步跳转到“**Charge** 充电详情页”，实时渲染当前的充电状态与数据；当充电完成或用户主动结束时，页面会跳转到最后的“**Register** 订单结算页”进行费用核算与支付，结算完成后支持一键返回主页，从而完成了一个闭环的充电生命周期。

第二个是“**Order** 订单页”。该页面主要负责用户资产与消费历史的管理，清晰地查看历史订单与进行中订单。它不仅方便用户进行历史回溯，还能让用户在退出充电详情页后，随时通过订单页重新找回当前正在进行的充电会话。

第三个是极具前瞻性的“**AiServer AI 助手页**”。这一板块充分结合了您整体系统中的边缘 AI 与云端智能，为用户提供智能问答交互。它能够根据小车回传的电池健康度、充电桩的实时负荷以及用户的充电习惯，给出个性化的充电建议或故障排查指导。

第四个是“**Me 我的页**”。作为常规的系统管理后台，这里聚合了用户的个人信息、系统通用设置以及充值功能，用于满足用户个性化配置的需求。

整款鸿蒙应用不仅模块划分清晰，而且在核心的充电操作链路上采用了多级“独立跳转”与“逐级返回”的路由设计。这种设计非常符合鸿蒙系统 **ArkUI** 声明式开发中对页面路由的管理规范，既保证了业务数据的单向高效流转，又为用户提供了丝滑、直观的操作体验。

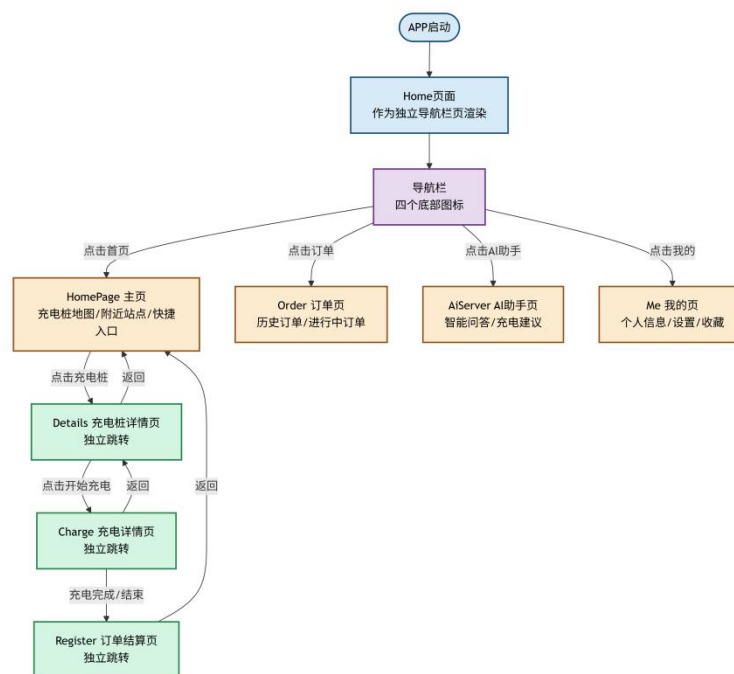


图 25 APP 软件架构

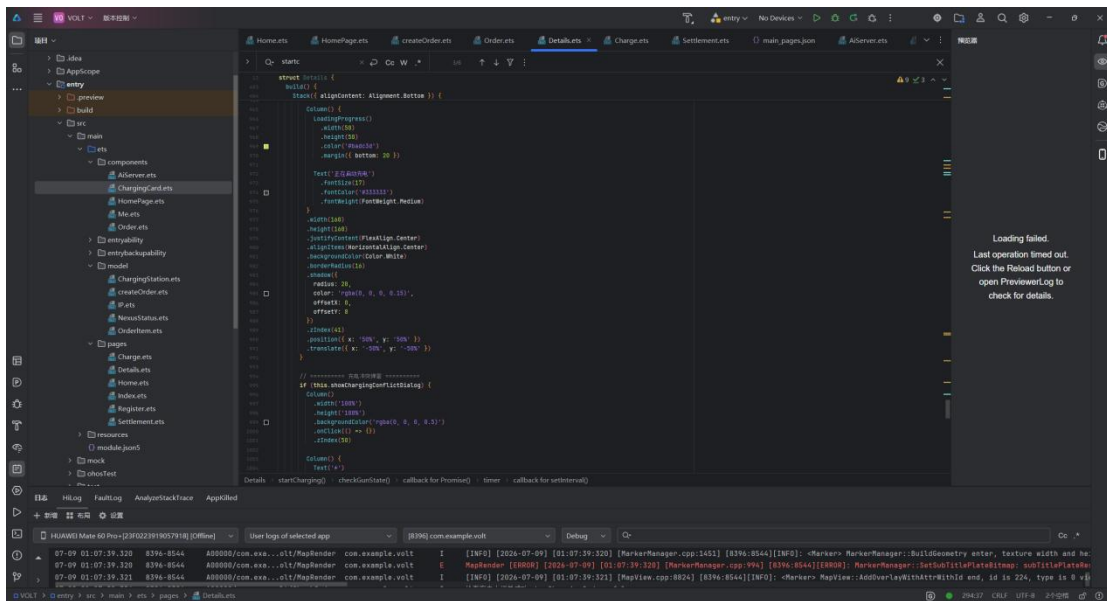


图 26 APP 代码截图

在整个云端的设计中，API 服务器扮演着面向用户端的业务门户角色。当用户通过鸿蒙应用进行刷新、查询、修改个人资料或查看历史订单等常规操作时，应用会向 API 服务器发起 HTTP 请求。API 服务器直接对底层的数据库进行读写操作，高效地为客户端提供增删改查（CRUD）的能力。当用户在应用上点击“开始充电”或“停止充电”等控制指令时，这些指令同样先通过 HTTP 协议送达 API 服务器，由其进行业务权限校验后，再向内部的通信总线进行关键的指令转发。

而云端与底层硬件的桥梁，则由独立的 MQTT 服务器来支撑。由于物联网硬件需要高频次、低功耗且长连接的通信流，充电桩在运行过程中会通过 MQTT 上报数据，将实时的电压、电流、温度等物理状态发送给 MQTT 服务器。服务器接收到这些高频数据后，会直接将其写入数据库，完成全量数据的持久化存储，这也是 API 服务器后续能够读取并呈现给用户的基础。

当需要向下控枪时，整条链路的流转也非常顺畅。API 服务器接收到应用下发的控制指令后，通过内部协议将其转发给 MQTT 服务器。随后，MQTT 服务器利用已经建立好的长连接管道，以 MQTT 下发的方式将控制指令精准推送

到指定的充电桩硬件端。这种“HTTP 入、MQTT 出”的异步双协议设计，成功将高并发的用户业务与高可靠的硬件控制分离开来，既保证了用户体验的流畅性，又确保了底层硬件控制的实时性与安全性。

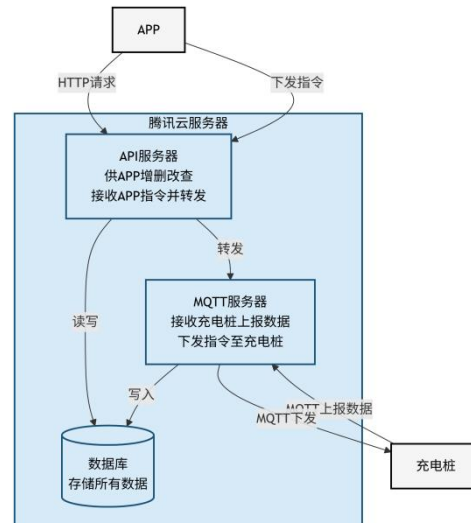


图 25 云服务器软件架构

```

131 gunNumber INTEGER NOT NULL,
132 starttime TEXT NOT NULL,
133 endtime TEXT,
134 endReason TEXT,
135 duration TEXT NOT NULL,
136 totalAmount INTEGER NOT NULL,
137 unitPrice INTEGER NOT NULL,
138 electricity INTEGER NOT NULL,
139 status TEXT NOT NULL CHECK(status IN ('待支付','已支付','充电中','已完成','已取消')),
140 statusColor TEXT
141 );
142 ---
143 ---
144 return [sql_user, sql_charging_station, sql_nexus_status, sql_order]
145
146 # ===== MQTT 核心库驱动 =====
147 mqtt_paho = mqtt_client.Client(callback_api_version=CallbackAPIVersion.VERSION2)
148
149 def update_station_status(gun1_state: str, gun2_state: str):
150     # 根据Gun1State和Gun2State更新充电桩状态
151     # 状态说明:
152     # - 空闲 (available): 00, 10, 02, 11, 21, 22, 20, 02
153     # - 占用 (occupied): 22
154     # - 故障 (broken): 其他 - 无效数据
155     ---
156     try:
157         # 转换字符串为整数
158         g1 = int(gun1_state) if gun1_state is not None else 0
159         g2 = int(gun2_state) if gun2_state is not None else 0
160     except (ValueError, TypeError):
161         print(f"Station Status 无效数据格式: Gun1={gun1_state}, Gun2={gun2_state}", flush=True)
162         return
163
164     # 判断状态
165     # 空闲: 00-02, 10-11, 21-22
166     if (g1 == 3 or g1 == 4 or g2 == 3 or g2 == 4):
167         new_status = 'broken'
168     else:
169         # 其他情况

```

图 26 云服务器代码截图

第三部分 完成情况及性能参数

3.1 整体介绍



图 27 充电桩正视图



图 28 充电桩斜俯视图

3.2 工程成果

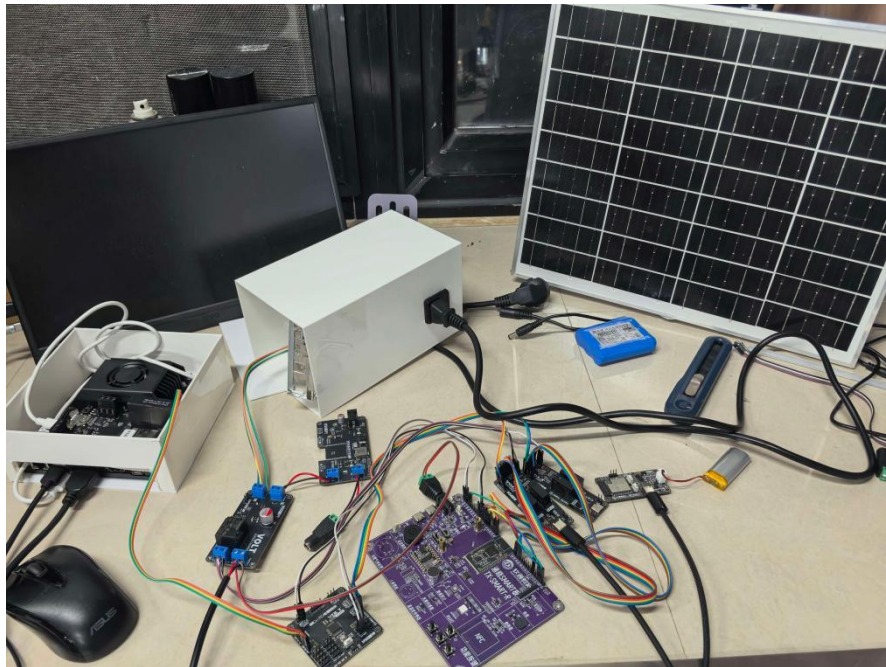


图 29 硬件电路连接图

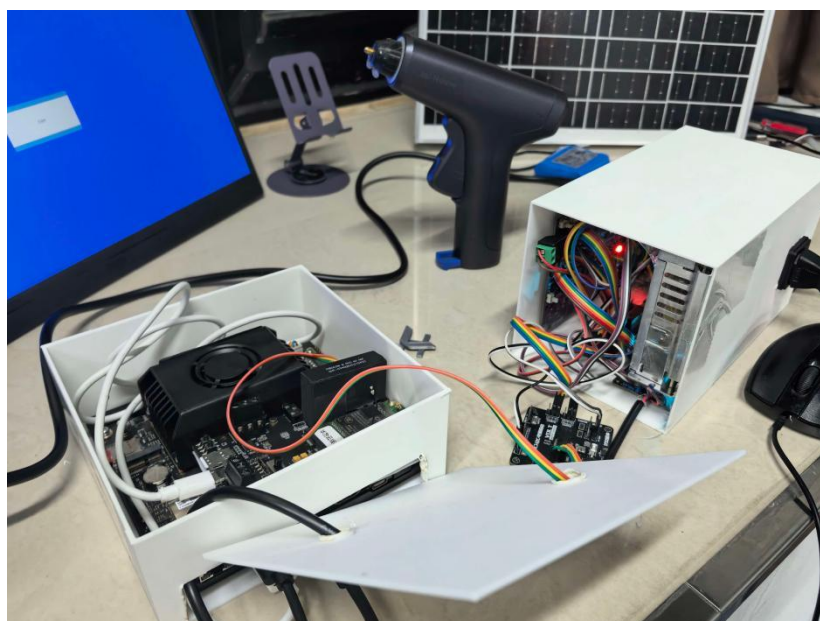


图 30 安装过程

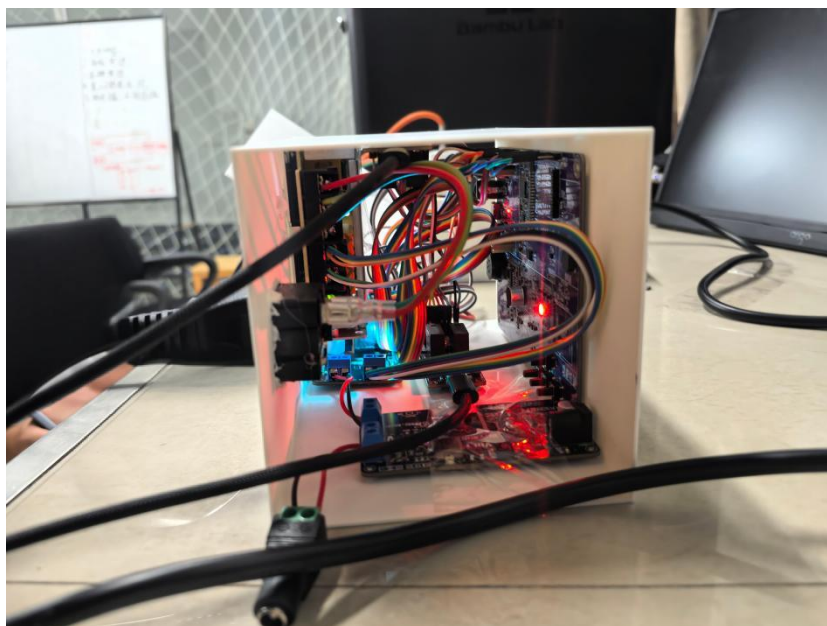


图 31 充电桩内部图



图 32 赛区赛版本充电桩和小车实拍图

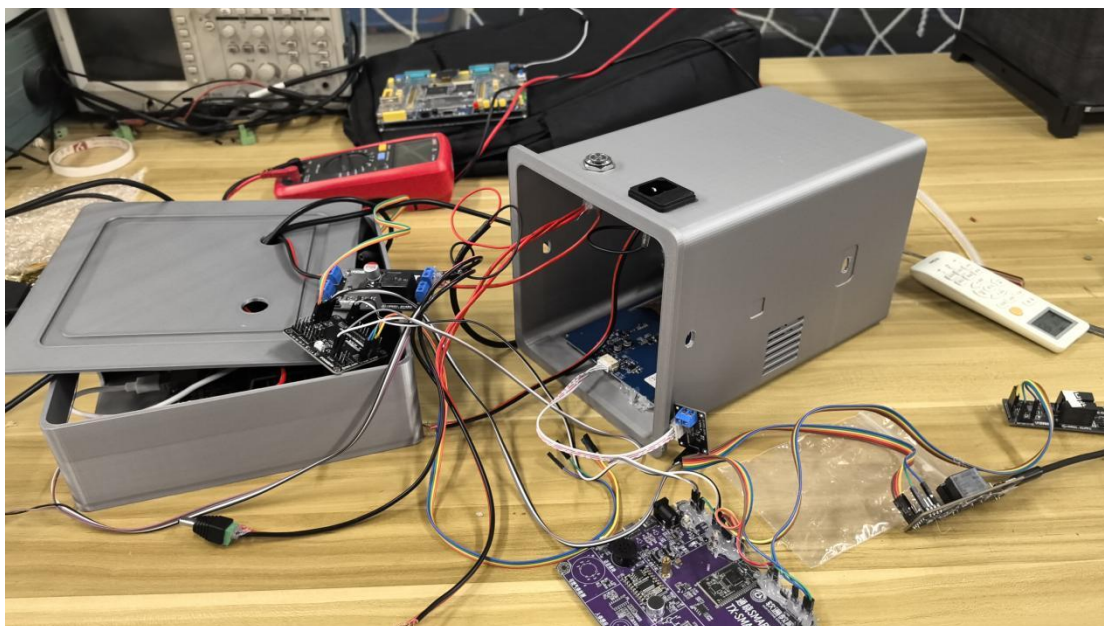


图 33 充电桩赛区赛版本内部图

第四部分 总结

4.1 可拓展之处

本项目由于过于庞大，且仅由一人开发，碍于时间关系，截至撰稿时，仍有多数功能正在在开发中。下一个版本，首先将增加触摸显示屏，使得充电用户可以在线下方便地人机交互。接着完善 APP 管理员后台，提供图表展示、数据统计、更高权限的数据编辑与设备控制。同时进一步完善光伏模块，使太阳能板支持跟随太阳转动，以及更全面的数据提供。最后还将利用起昇腾的算力，构建火灾检测、车牌检测等边缘计算功能。并基于使用到的广和通 L610 将火灾识别的画面实时传输视频流至手机上。

4.2 心得体会

从项目开始时的满腔热血，到队友摆烂背刺的咬牙切齿，再到我命由我不由天的不服输心态。一个人，从硬件，到嵌软，到 Linux，到前后端，再到建模，到组装。一个人，追星捧月，一步一步地将这个项目逐渐落地，看着他逐渐成型的样子，心里那股气也逐渐平复，不再去愤慨，逐渐去感恩上天给了那么一个锤炼自我的机会。一学期时光飞逝，读过的文档踩过的坑，使我的技术有了日新月异的变化。成绩固然已经不重要了，一个人泡在实验室沉浸在开发里的这半年，将是我一辈子难以忘怀的宝贵记忆。

第五部分 参考文献

[1] 薛小玲. OpenHarmony 嵌入式系统原理与应用——基于 RK2206 芯片[M]. 北京: 清华大学出版社, 2025.

[2] 华为技术有限公司. ArkTS 官方开发文档[EB/OL]. [2026-07].
<https://developer.huawei.com>.